App Attest: story, adversary, threat model

2026-10-19

Contents

1. Story 1
2. System model 2
3. Assets 2
4. Adversary 3
5. Trust assumptions 3
6. Enforcement 3
7. The free experiment: provenance 3
8. Failure classes (App Attest) 4
9. SDK-mediated adoption 5
10. Research questions 5
11. Methodological contributions 6
12. Scope statement 6
13. Open items 6

Working draft for the paper's framing. Scope is App Attest only; DeviceCheck enforcement classes (F6 to F8) are out of scope and stated as future work. Companion documents: `threat-model.md` (assets A1 to A4, loss model, ethics) and `system-model-figure.tex`.

1. Story

App Attest lets an app prove to its own server that a request came from an unmodified App Store build on a genuine device. Apple is explicit that the client cannot enforce anything: the server must verify the attestation at enrollment and every assertion afterwards, and the verification steps

are specified, including that the relying-party hash must match the developer's own App ID. The question nobody has measured is whether servers do this.

We take a modified client, the thing the mechanism exists to reject, and ask each server to serve it. The result is a tiered picture: servers that reject the modified client at the door, servers that let it enroll but hold the line on assertions, and servers that adopted the API and enforce nothing. The failure taxonomy gives each gap a name, the loss model says what it costs, and the adversary-cost measurement says how cheap the gap is to exploit.

Three things make this credible rather than anecdotal:

1. corpus scale, with a static-to-dynamic funnel and the funnel's losses reported;
2. tri-state verdicts, with the inconclusive mass reported by reason;
3. the confounds that would inflate enforcement, above all the re-sign entitlement crash, are neutralized and quantified.

One-line thesis: *attestation is adopted far more than it is enforced, and the gap is exploitable by a client-side adversary with no jailbreak.*

2. System model

component	role
Client app	calls `DCAppAttestService`; the adversary's binary in our model
Apple attestation service, Secure Enclave	issues the key, signs attestations and assertions; trusted
Developer server	verifies; the object of study
Relying party	the developer's App ID: team identifier plus bundle identifier

Enrollment: `generateKey`, then `attestKey` over a server challenge. The server verifies the attestation object: certificate chain to Apple's root, nonce, relying-party hash, counter zero, environment.

Use: `generateAssertion` over `clientDataHash`, derived from the request and a server challenge. The server verifies the signature, a strictly increasing counter, challenge freshness and request binding.

From iOS 27 the authenticator data also carries the launch category (App Store, TestFlight, other) and the bundle version: two more provenance signals a server may read.

3. Assets

Carried over from `threat-model.md`:

- **A1** revenue-bearing state: purchases, entitlements, credits, trials
- **A2** abuse limits: rate limits, promo and referral caps, bans
- **A3** request integrity: scores, votes, parameters, prices

- **A4** API access: scraping, automation

4. Adversary

A client-side adversary with full control of one app instance and none over Apple or the Secure Enclave.

capability	description
C1 modified client	re-sign and run an altered build, inject code, hook the attestation API
C2 own-traffic control	read and alter requests leaving their own device (user-installed CA, proxy, hooks at the URL-session boundary)
C3 replay	resend captured assertions and requests
C4 scale	repeat across devices and accounts

Cannot: forge Secure Enclave signatures, obtain Apple's attestation root, or touch the server-to-Apple path.

No jailbreak is assumed. Re-signing, dylib injection and a user-installed CA are the whole kit. This is also the adversary Apple's own documentation describes the mechanism as defending against (re-signed with a non-matching provisioning profile, cheat menus, falsified requests).

5. Trust assumptions

- Apple's attestation service and the Secure Enclave are trusted.
- The app binary is untrusted: it is the adversary's.
- The developer's server is the object of study.
- The client-side network is adversary-controlled; the server-to-Apple path is not.

6. Enforcement

A server **enforces** App Attest if a request for a protected resource that lacks a valid, fresh, request-bound assertion from a key enrolled under the developer's own relying party is rejected.

Soft enforcement: the server detects the violation and degrades the response (limited data, heightened monitoring) instead of blocking. Apple recommends this. It is reported as its own class, never folded into fail-open.

7. The free experiment: provenance

Our client is re-signed under the lab team, so every attestation it produces carries a foreign relying-party hash and, on iOS 27, a non-App-Store launch category. A server that implements Apple's verification refuses it at enrollment. The baseline run is therefore the strongest probe and sorts servers into tiers before any probe script loads.

tier	observed	meaning	classes
T0	enrollment upload rejected; gated data never served to the modified client	provenance verified	enforced per specification
T1	modified client enrolls; omit, tamper, bind and replay all refused	signatures verified, provenance not	F5
T1s	as T1, but tampered probes receive a degraded response	soft enforcement	F5, soft
T2	tampered or missing assertions served in full	adopted, not enforced	F1, F3, F4 on top of F5
T3	API called, but no assertion ever leaves the device with a data request	adopted, unused	F2-adjacent

Every app that reaches the probe battery has already failed F5 in the strict sense. This is the headline.

Tooling: the baseline grades an attestation-object upload that returns a non-2xx, with no gated data served, as T0, and confirms it with a second baseline run. Login failures are separated by the login driver's status; re-sign entitlement crashes by the shim and the classified crash reason; unrelated refusals (version gates, rate limits) by their wording. A T0 or T3 baseline ends the run: no probe is armed. The default battery for the rest is five cycles (baseline, omit, tamper, bind, replay).

8. Failure classes (App Attest)

class	failure	probe / signal	status
F1	adopt, never verify: served without a valid assertion	`omit` (assertion absent, judged on value), `tamper` (real assertion, signature bytes corrupted)	measured
F2	enrollment verified, per-request assertions not	traffic shape only (attested enrollment, unattested data requests)	flagged, not probed; future work

class	failure	probe / signal	status
F3	assertion not bound to the request	`bind` (request altered at the URL-session boundary after signing, gated endpoints only)	measured
F4	no counter or challenge enforcement	`replay` (first assertion re-served on later calls)	measured
F5	provenance and environment not validated	T0 versus T1 from the baseline; statically, the App Check debug provider and the environment entitlement	measured via tiers

Soft enforcement is detected by diffing each probe response against the baseline response for the same endpoint (status class, size, JSON key overlap).

Coverage is classified per app: per-request, per-session (attest at login, then a credentialed session), partial (unattested data endpoints not explained by a session), none. Only partial counts against the app.

9. SDK-mediated adoption

For Firebase App Check the provider performs Apple's verification, so a re-signed client cannot obtain a token at all. Enforcement collapses to one question: does the backend require the token, which Firebase leaves off by default. This yields a separate adopt-versus-enforce number for the SDK-mediated population. The static stage records the SDK and the provider (App Attest, DeviceCheck, debug); a debug provider in a shipping binary is a finding on its own.

10. Research questions

- **RQ1 adoption.** Direct versus SDK-mediated, production versus development environment, by category, and whether the assertion ever travels with a data request.
- **RQ2 enforcement.** Tier distribution and class distribution among adopters, hard and soft separated, inconclusive mass reported with reasons.
- **RQ3 what is protected.** Gated endpoints classified against A1 to A4; this connects a tier to a loss.
- **RQ4 adversary cost.** For each T1 or T2 app: nothing, a re-sign, a pinning bypass, or an unbroken hard pin (from the pinning stage's evidence).
- **RQ5 case studies.** One per class where the gated asset is revenue or integrity, with responsible disclosure.

11. Methodological contributions

- The re-sign as a provenance experiment (T0 for free).
- The entitlement confound: a re-signed App Store binary can crash at launch from a stripped entitlement; naive measurement reads this as enforcement. The shim neutralizes it and the crash reason is classified, so the artifact is quantified rather than counted.
- Tri-state grading with inconclusive-by-reason at every stage.
- Finding attested requests by content fingerprint (the CBOR prefix of an assertion or attestation object) and by hook-to-request timing, independent of header names.

12. Scope statement

DeviceCheck enforcement (F6 to F8) is reported as usage only: how many apps call `generateToken`, where in the flow, and whether the token travels with a gate request. Enforcement of DeviceCheck-backed gates is per-app scenario work with unresettable device state and is left to future work, with at most a case study.

13. Open items

1. Baseline grading for T0 versus login failure versus entitlement crash: implemented (`attestation.classify_baseline`, confirmed by a repeated baseline; T3 and runtime non-use are graded the same way).
2. F2 probe: an unregistered key (suppress or corrupt only the attestation upload, then assert with that key).
3. On-device validation of the signature-only tamper, bind and replay scripts.
4. Formal statement of the enforcement predicate (security game or predicate), matching the rest of the paper.